Habla con tu dinero — Memoria técnica

Cátedra Unicaja · Universidad de Granada

Jose Angel Carretero Montes

Octubre 2026

1. Qué es el proyecto

"Habla con tu dinero" es un asistente financiero conversacional: el usuario escribe en lenguaje natural ("¿cuánto gasté en restaurantes este mes?", "manda 30€ a María") y la aplicación responde con datos reales de su cuenta, gráficos dinámicos y, cuando la petición implica mover dinero, una propuesta que el propio usuario debe aprobar antes de que se ejecute. El objetivo es explorar cómo dar a un LLM capacidad de actuar sobre datos financieros reales sin perder el control ni la seguridad que exige ese dominio.

2. Subsistemas, a grandes rasgos

- **Chat financiero con IA.** Un agente orquestado con LangGraph decide, mediante *tool calling*, qué consultas hacer sobre la base de datos del usuario (saldos, gastos por categoría, inversiones…) y redacta la respuesta final en streaming.
- **Operaciones validadas por código, no por el LLM.** Cuando la petición implica mover dinero (Bizum, inversión), el modelo solo *propone*; un segundo paso determinista valida fondos y límites, y el usuario aprueba o rechaza en un modal antes de que nada se ejecute en la base de datos.
- **Dashboard proactivo.** Cada noche, un job genera tarjetas personalizadas (avisos de gasto, oportunidades de inversión, progreso de metas) a partir del contexto financiero real de cada usuario, sin que este tenga que pedirlas.
- **Memoria de usuario (RAG casero).** El historial de chat se analiza para extraer hechos y preferencias ("prefiere ahorrar antes que invertir"), que se recuperan por similitud y se inyectan en el prompt de conversaciones futuras.
- **Modo voz.** Transcripción y síntesis de voz locales (sin servicios externos de pago) para interactuar hablando en vez de escribiendo.
- **Historial de conversaciones.** El usuario puede volver a abrir cualquier conversación anterior, agrupada por día.

3. Arquitectura

El frontend (React) habla con el backend (FastAPI) por HTTP/SSE con autenticación JWT. El backend delega la orquestación de IA en LangGraph, que a su vez usa **litellm** como capa de abstracción sobre el proveedor de modelo configurado (Anthropic Claude por defecto; también Gemini, Groq, Mistral u Ollama local, sin cambiar una línea de código). Los datos viven en SQLite a través de SQLAlchemy.

4. Modelo de datos

Diez tablas: usuarios, cuentas, transacciones, inversiones, transferencias Bizum, acciones pendientes de aprobación, metas de ahorro, alertas del dashboard, conversaciones de chat y los vectores de memoria de usuario.

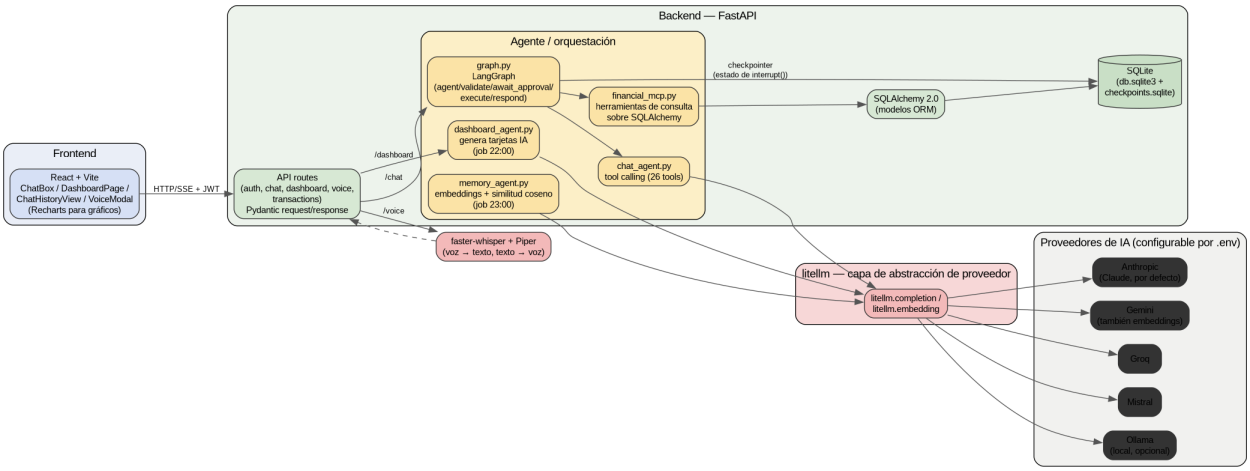


Figure 1: Arquitectura general

5. El grafo de decisión del agente

El punto más delicado del diseño es que **el LLM nunca mueve dinero directamente**. El grafo de LangGraph separa explícitamente la parte "creativa" (qué quiere hacer el usuario, cómo se redacta la respuesta) de la parte crítica (¿hay saldo?, ¿está dentro de los límites?, ¿lo aprobó el usuario?), que es código determinista normal, testeado como cualquier otra función.

El nodo `agent` solo puede *proponer* una acción mediante tool calling; `validate` comprueba fondos y límites sin ninguna llamada al LLM; `await_approval` pausa el grafo con `interrupt()` (el estado queda persistido en SQLite) hasta que el usuario decide; y `execute` aplica el cambio en una única transacción, de forma idempotente (reintentar no puede duplicar un movimiento). Si en cualquier punto la validación falla o el usuario rechaza, se pasa directamente a `respond` sin tocar la base de datos. Este determinismo es la garantía de que ninguna alucinación del modelo puede traducirse en un movimiento de dinero no autorizado.

6. Stack tecnológico

Capa	Tecnología
API	FastAPI (async)
Validación de datos	Pydantic v2
ORM / Base de datos	SQLAlchemy 2.0 + SQLite
Autenticación	JWT
Orquestación de IA	LangGraph
Modelo de lenguaje	litellm (Claude por defecto; Gemini/Groq/Mistral/Ollama soportados)
Memoria / RAG	litellm embeddings + similitud coseno
Voz	faster-whisper (STT) + Piper (TTS)
Frontend	React + Vite
Gráficos	Recharts
Contenedores	Docker Compose

Documentación técnica completa, capítulo a capítulo: `docs/README.md`.

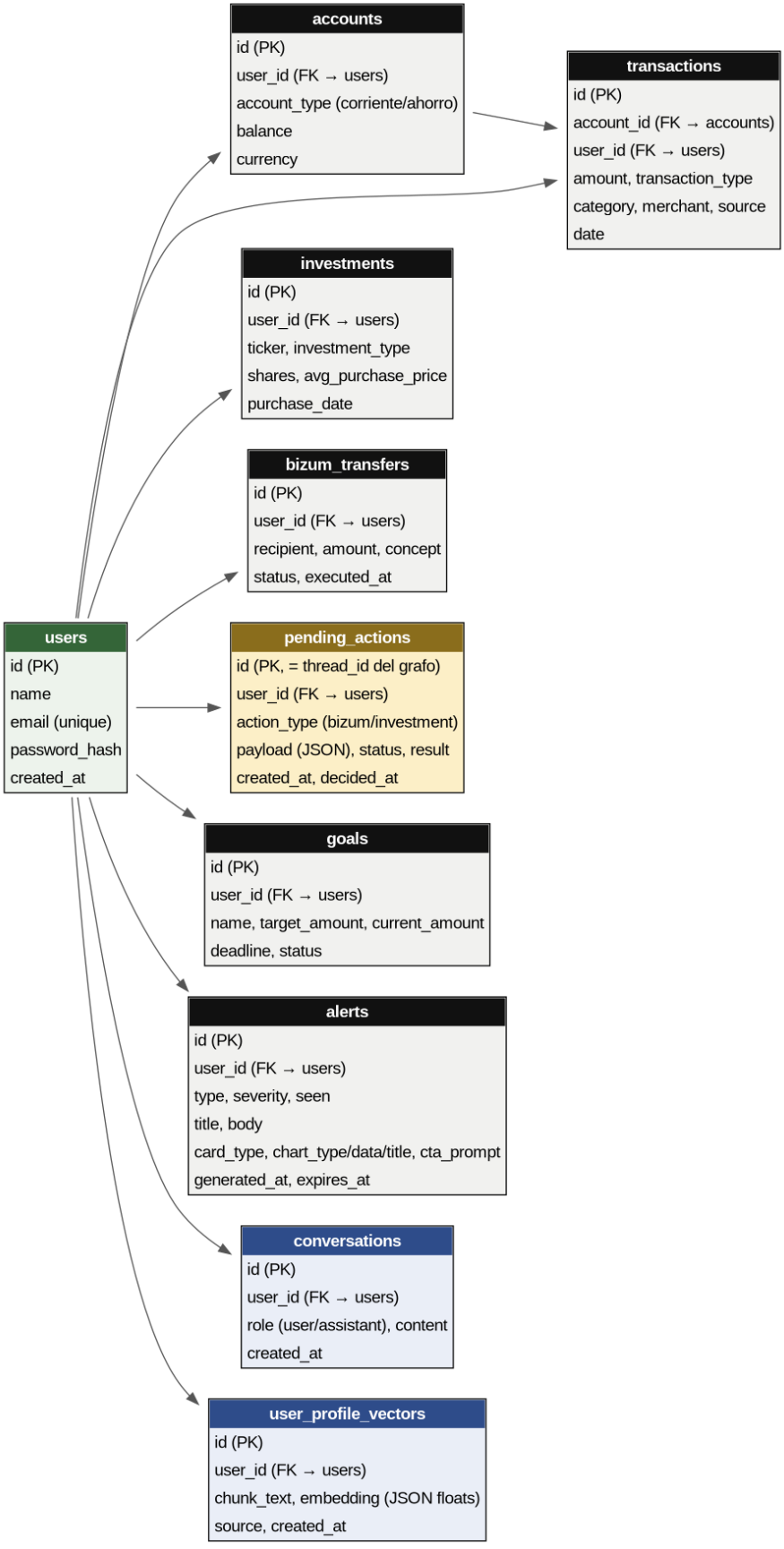


Figure 2: Esquema de la base de datos

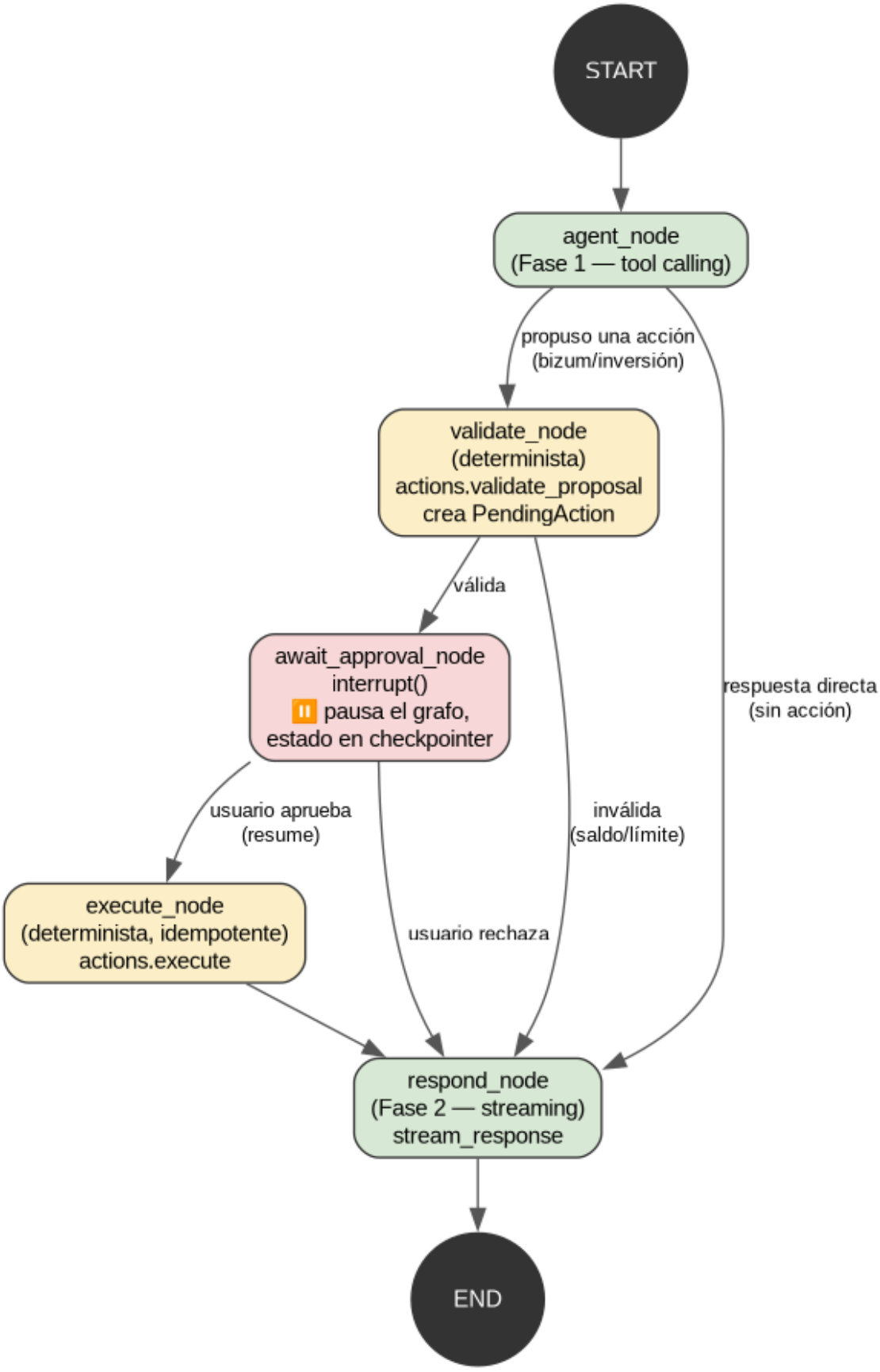


Figure 3: Grafo de LangGraph